

Molecular Dynamics Simulation of Argon

Mankrit Singh, Noa Aarts, Raghav Juyal

March 2025

Abstract

In this project, we implemented a molecular dynamics simulation of Argon particles using the Lennard-Jones potential, employing the Velocity-Verlet algorithm for numerical integration. Once the system reaches the desired temperature at equilibrium, the simulation tracks various observables such as Mean Squared Displacement (MSD) and Specific Heat per atom to analyze the system's behaviour across different thermodynamic conditions. A phase diagram was constructed by varying temperature and density, with automated phase classification using curve fitting. To ensure efficient execution, we profiled our code, optimised the time-consuming sections, and used caching techniques. This report details our implementation choices, performance optimisations, results and the insights gained from our results.

1 Introduction

Understanding the behaviour of materials under different thermodynamic conditions is fundamental to condensed matter physics and statistical mechanics. Argon, a noble gas, serves as a well-studied model system for investigating phase transitions due to its simple interatomic interactions governed by the Lennard-Jones potential.

Molecular dynamics simulations allow us to track the time evolution of a system of particles by integrating Newton's equations of motion. The choice of numerical integrator is crucial for ensuring the accuracy and stability of the simulation. While simpler methods like the Euler integrator introduce significant energy drift, we employed the Velocity-Verlet algorithm, which offers better energy conservation. In order to ensure that the simulation achieves the desired temperature at equilibrium, we use velocity rescaling.

In this study, we examine the thermodynamic properties of Argon under varying conditions. We measure observables such as the Mean Squared Displacement (MSD) and Specific Heat per atom. MSD is used to characterize phase behaviour, and we estimate errors for the Specific Heat per atom using the block bootstrap method. Using these results, we construct a phase diagram by automatically classifying phases based on curve-fitting techniques. Given the computational expense of such simulations, we also explore performance optimizations, including multiprocessing and caching, to reduce redundant computations.

The structure of this report is as follows: Section 2 begins with some relevant background information for the simulation, and then proceeds with detailing the choice of integrator and methodology for estimating the phase of Argon. It concludes with some comments on performance and error estimation. Section 3 showcases the results we obtained for the phase diagram and specific heat per atom observable, which is followed by the Conclusion (Section 4) of the report.

2 Methods

In this chapter, we discuss the structure of the simulator and the prerequisite background knowledge. We start by talking about the various constraints that the simulator should have in section 2.1, followed by details of the usage of dimensionless units in 2.2. We continue in section 2.3 where we talk about the integrator we build for this simulation and then section 2.4 elaborates on velocity rescaling. Then,

section (2.5) discusses the displacement of individual atoms over time. In section 2.6 we examine using the results from the mean squared displacement to estimate the phase of the system and how we made the wait as short as possible in section 2.7. Finally, we have a look at how we estimate the specific heat and its error estimation in section 2.8.

2.1 Boundary conditions

Before we can talk about our simulator, we should discuss the conditions and approximations we used. Since we can't simulate the Schrodinger equation for billions and billions of particles in even a milliliter of material we have to make some approximations. The first approximation we use is that the particles are perfectly localized, this means that we can know both the position and momentum perfectly and that the particles don't act as waves. We also approximate the potential between any two particles using the Lennard-Jones potential, shown in Eq.1, we received in the lecture notes[1, week 1].

$$U_{LJ}(r) = 4\epsilon\left(\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^6\right) \quad (1)$$

Both of these approximations allow us to simulate more particles but not nearly enough to simulate the bulk of the material. This brings us to a third approximation, the periodic boundary condition. In this condition we can define a box where if you move out of a side, you instantly appear at the opposite side. This means that the particles "see" an infinite space they can move around in and also infinite copies of themselves and all the other particles. That's a problem since calculating the total potential would involve summing over infinite pairs of particles. We need to use yet another approximation, the minimal-image convention. The convention is based on the fact that for large r we get $U_{LJ}(r) \approx 0$. We can therefore only consider the single nearest copy of the other particle when calculating the effective potential. After applying all the approximations, we can find the equations of motion which we see in Eq. 2.

$$\frac{d^2\mathbf{x}}{dt^2} = \frac{\mathbf{F}(\mathbf{x}(t))}{m} \quad (2)$$

where $\mathbf{x}(t)$ contains the positions of all the particles and $\mathbf{F}(\mathbf{x}) = -\nabla U_{LJ}(\mathbf{x})$. Finally, time is sadly not discrete in this universe and we can't solve the equations of motion analytically for more than 2 particles [2]. Therefore we have to use a numerical integrator with a discrete step size τ which will be our final approximation.

2.2 Dimensionless Units

To make our simulation more generalised and to not keep track of nasty constants, we made use of dimensionless units based on the characteristic values of Argon. By rescaling our physical quantities, we reduce the dependence on unit-specific values, allowing the simulation to be applicable to a broader range of atomic systems.

The units for some of the important parameters of our simulation are listed below:

$$r = \tilde{r} \cdot \sigma \quad (\text{Position}) \quad (3)$$

$$U = \tilde{U} \cdot \epsilon \quad (\text{Energy}) \quad (4)$$

$$t = \tilde{t} \cdot \sqrt{\frac{m\sigma^2}{\epsilon}} \quad (\text{Time}) \quad (5)$$

$$F = \tilde{F} \cdot \frac{\epsilon}{\sigma} \quad (\text{Force}) \quad (6)$$

$$T = \tilde{T} \cdot \frac{\epsilon}{k_B} \quad (\text{Temperature}) \quad (7)$$

$$\rho = \tilde{\rho} \cdot \frac{N}{\sigma^3} \quad (\text{Density}) \quad (8)$$

$$(9)$$

Here, the constants ϵ and σ are taken from the Lennard-Jones potential (Eq. 1), m is the mass of the particle, k_B is the Boltzmann constant, and N is the total number of particles.

All quantities with a $\sim$ on top are dimensionless units, which were then utilised in their corresponding modified formulae.

2.3 Velocity-Verlet Integrator

For a physics simulation, it is essential that energy is conserved, we discuss why the Velocity-Verlet integrator does this well. The Euler integrator, which was the first integrator we built, uses repeated applications of Eq. 10 on the initial state to get the time progression. As shown in Fig. 1 the Euler integrator does not conserve energy well.

$$\begin{aligned}\mathbf{x}(t + \tau) &= \mathbf{x}(t) + \tau \mathbf{v}(t) \\ \mathbf{v}(t + \tau) &= \mathbf{v}(t) + \tau \mathbf{F}(t)\end{aligned}\tag{10}$$

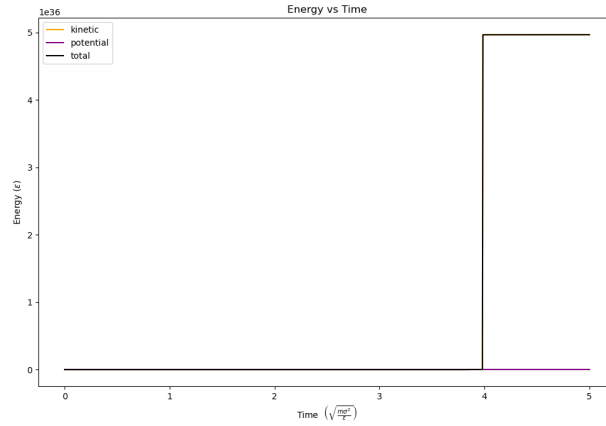


Figure 1: The energies of the system for the Euler integrator over time. The black line should have been completely flat if the integrator was perfect. To reproduce this figure and figure 2 use config 5.1.1

with `main.py`

We use the fact that the equation of motion is a second-order differential equation. Using the Taylor expansion of this equation, and taking the first and second order terms brings you to the Verlet algorithm as shown in the lecture notes[1, week 3], since we also have the velocities of the particles we can use the Velocity-Verlet algorithm instead shown in eq. 11.

$$\begin{aligned}\mathbf{x}(t + \tau) &= \mathbf{x}(t) + \tau \mathbf{v}(t) + \frac{\tau^2}{2} \mathbf{F}(\mathbf{x}(t)) \\ \mathbf{v}(t + \tau) &= \mathbf{v}(t) + \tau \frac{\mathbf{F}(\mathbf{x}(t)) + \mathbf{F}(\mathbf{x}(t + \tau))}{2}\end{aligned}\tag{11}$$

The implementation we've made can be found in this part of the code. If we simulate the same initial state as we tried with the Euler integrator, we can see in Fig. 2 that the total energy stays pretty much constant when we use the Velocity-Verlet integrator. Now that we have a simulation that conserves energy, we are one step closer to calculate various observables on the data it generates, but before that, we need to ensure that our system actually equilibrates to the desired temperature.

2.4 Velocity Rescaling

Before we proceed with computing the observables, we need to ensure that the system has an equilibrium temperature of what we desire to simulate. To achieve this, we make use of the velocity rescaling

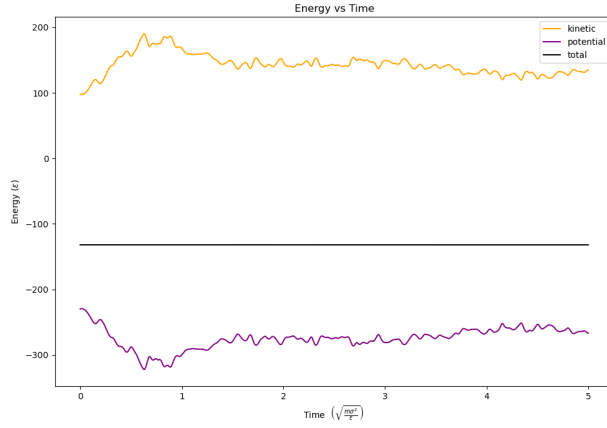


Figure 2: The energy of the system for the Velocity-Verlet integrator for the same initial state as figure 1. As we can see the total energy (black) stays much more constant as it should.

method mentioned in the lecture notes [1, week 4]. In practice, this means that we perform periodic checks and apply rescaling when we observe that the temperature of the system is not near the desired temperature up to a tolerance level. Once the temperature has not rescaled for a certain number of checks, we stop rescaling for all future timesteps and note this time as the equilibrium timestep.

$$v_{i,new} \rightarrow \sqrt{\frac{3(N-1)T}{\sum_i v_{i,old}^2}} \cdot v_{i,old}$$

The above equation is written in dimensionless units, where v_i refers the old / new velocity for the particle, N refers to the total number of particles and T is the desired temperature of the system.

Once desired equilibrium temperature is achieved, we can proceed towards calculating the observables.

2.5 Mean Squared Displacement

One of the observables we wanted to find was the Mean Squared Displacement or MSD. Each particle will move around space due to the forces of the other particles, the position relative to its starting point is called the displacement. We want to calculate the MSD by first finding the size of all the particle displacements and then taking the average. After that it's possible to plot these MSDs at various times t .

In a study by E. Zohravi[3] it is said that for a solid, the MSD should stay mostly constant after an initial jump due to the particles being localized while in a liquid or gas, it should continue increasing either linearly or exponentially as shown in 3. We found that our simulation didn't reproduce the expected MSD for a liquid when watching the animation qualitatively showed us that there was a liquid.

Later we found that the periodic boundary condition on the particle positions was the issue, after fixing the saved positions to not have the periodic condition applied, while making sure the forces still got calculated correctly, we get Fig. 5. Looking at the included fit it has an exponent of ≈ 1 which aligns very well with our expectation for a liquid.

2.6 Phase Estimation

By changing the size of the box we were able to get MSD plots of three distinct shapes as shown in Fig. 6. These figures also include a fit which was generated using `scipy.optimize.curve_fit` of a

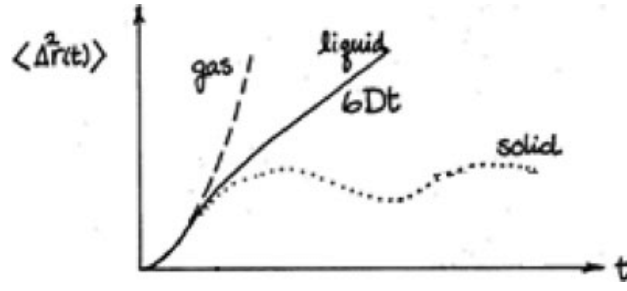


Figure 3: Typical mean-squared displacement depicted for gases, liquids and solids.[3, fig. 1]

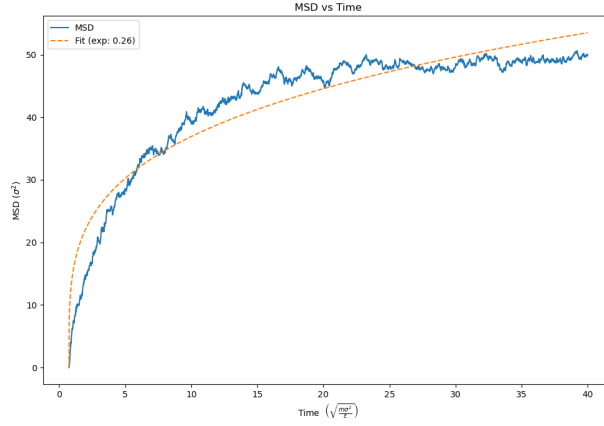


Figure 4: The calculated MSD for a liquid that doesn't align with the literature. Generated using a slightly modified simulation with config 5.1.2

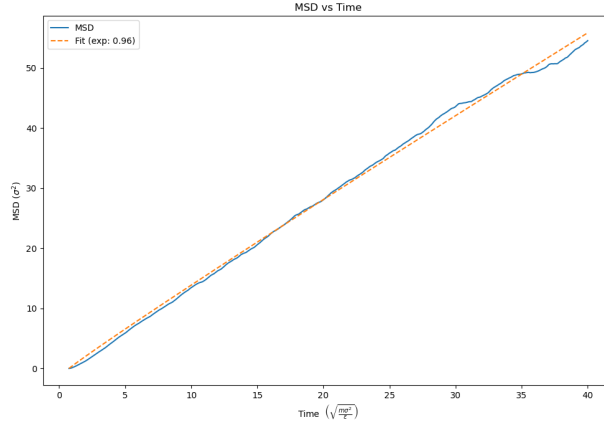


Figure 5: The calculated MSD for a liquid that does align with the literature. Generated using config 5.1.2

function like Eq. 12 where A and B are parameters that need to be optimized.

$$f(t) = At^B \quad (12)$$

This allowed us to use the exponent term B to retrieve the phase of the argon, if $B \approx 0$ the argon would be solid, if $B \approx 1$ it is a liquid and finally if $B \approx 2$ the argon is in the gas phase. We wrote a second

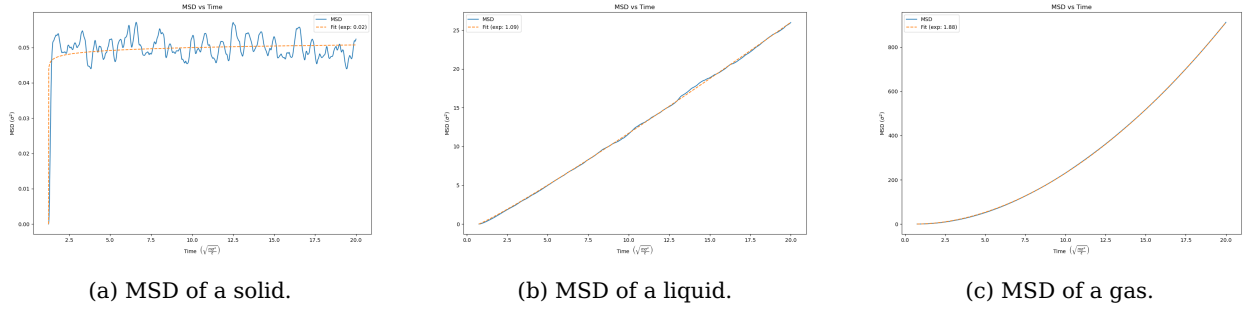


Figure 6: Generated using configs 5.1.3

script to run many simulations at different box sizes and temperatures with the goal to create a phase diagram like the one shown in Fig. 7 from simulation data.

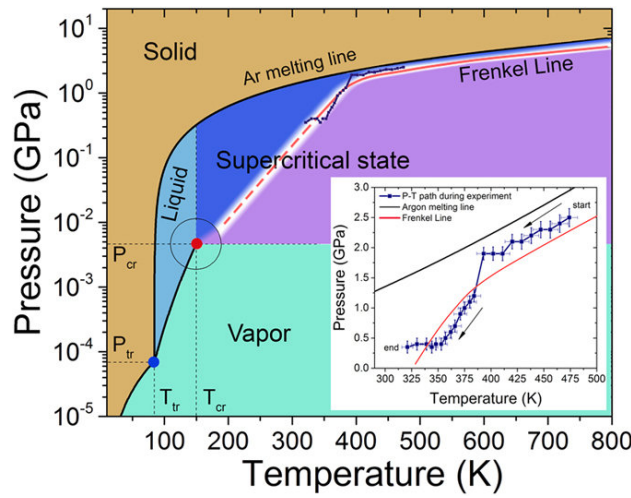


Figure 7: A phase diagram of argon out of a paper by D. Bolmatov[4]

2.7 Performance

While we thought the simulator was already quite performant, simulating 500 particles at ≈ 29 steps per second on a single core. The creation of the ≈ 1700 data-points for the phase diagram, each a simulation of 500 particles for 4000 steps, took a total of 17 hours.

Sadly we couldn't improve the simulation a lot without more approximations or using faster libraries, we were spending nearly all of the time in various numpy functions. Therefore ensuring that simulations that already happened were reused for subsequent runs was paramount. This is why we added caching using python's pickle files, using this cache we could be sure that if some part late in the creation of the phase diagram crashed the program only a small amount of progress would be lost.

We also attempted to use cupy[5] instead of normal numpy. This would have allowed the calculations to happen on the GPU instead of the CPU and would probably have significantly sped up the simulator, but due to time constraints we weren't able to fully finish this, the progress we've made so far can be found here.

While the large amount of simulations we performed for the phase diagrams took a lot of time, the combination of python's multiprocessing library and the caching we were able to run what we wanted to run and iterate on the visualization quickly afterwards due to the cache.

2.8 Specific Heat

Another observable that we wanted to calculate was the specific heat capacity for Argon, which provides insight into the thermodynamic behaviour of the system. The specific heat, denoted as C_V , quantifies how much energy is required to change the temperature of the system. This is calculated using the following expression:

$$C_V = \left(\frac{2}{3N} - \frac{\delta K^2}{\langle K \rangle^2} \right)^{-1}$$

Here, K refers to the Kinetic Energy and δK^2 is the variance of the Kinetic Energy, while N is the number of particles. Ultimately we use $c_v = C_V/N$ as the specific heat per atom as our observable, which is referenced in the lecture notes [1, week 4].

2.8.1 Error Estimation

To estimate the error in specific heat per atom, we use the block bootstrap method. First, we take the kinetic energy values after equilibrium. We use a block size of 100 and compute the block means of kinetic energy and the square of kinetic energy. We then bootstrap the blocks, making sure to use the corresponding block for both kinetic energy and its square, compute the specific heat per atom and repeat. This sampling process is performed 10^4 times. We then compute the mean of specific heat per atom and compute the error using the following:

$$\sigma_{c_V} = \sqrt{\langle c_V^2 \rangle - \langle c_V \rangle^2} \quad (13)$$

3 Results

3.1 Phase Diagram

To get the phase diagram we need to vary pressure and temperature. Since we do not know the pressure beforehand, we vary the density of particles. This varies the volume of the box which effectively changes the pressure of the system. Inspired by the ideal gas equation, we figured that the relation between pressure and density is as follows:

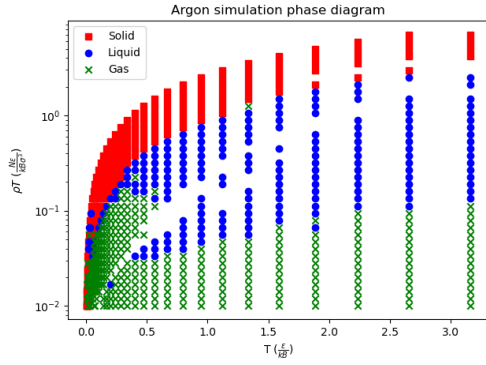
$$P \propto \rho T \quad (14)$$

Thus, we varied ρT logarithmically from 10^{-2} to 10^1 and varied T logarithmically from $10^{-2.5}$ to $10^{0.5}$ for 41 values each. More details about the other parameters can be found in 5.1.4. This gives us the plots in 8. While we can see the similarities to the actual plot 7, there are some errors. Firstly, some points do not reach equilibrium in the timesteps that we provide. Secondly, for some points, the energy blows up. These cases were excluded from the plots. We suspect that due to not simulating for long enough, the mean square displacements of some cases are difficult to differentiate especially for liquids and gases.

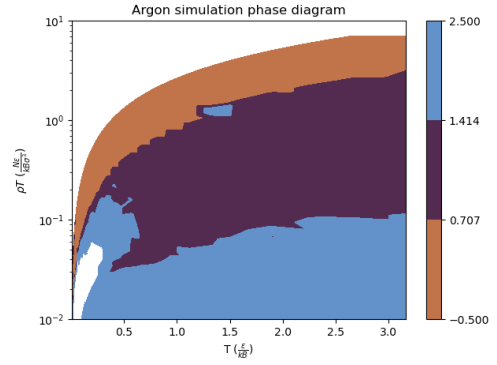
3.2 Specific Heat Per Atom

Once we got the phase diagram plots, we searched for regions of solid, liquid and gas so that we see how specific heat per atom varies with temperature for the different phases. For solid, we used $\rho T = 3$, for liquid, we used $\rho T = 0.5$ and for gas, we used $\rho T = 0.01$. For all 3 we varied temperature linearly from 1.75 to 2 for 10 values. More details about the other parameters can be found in 5.1.4. This gives us figures 9 and 10.

For solid, from 10a, we see that the values are close to the value predicted by Dulong-Petit law, i.e., $c_V = 3$ [6]. For gas, from 10c, we see that the values are close to the theoretical value of monotonic gas, i.e. $c_V = 1.5$ [6]. We see from 10b that it seems to decrease with the increase of temperature which seems to match what we see in 11. It seems like we are beyond the critical temperature but we



(a) Scatter Plot



(b) Contour Plot

Figure 8: 8a is the scatter plot and 8b is the plot generated by interpolating the scatter plot values using `scipy.interpolate.griddata`

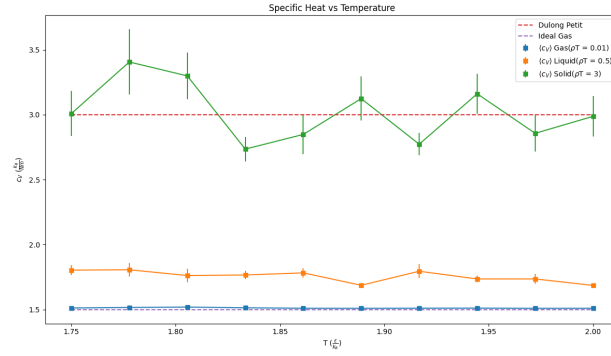
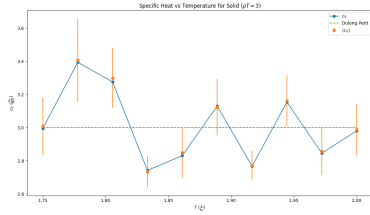
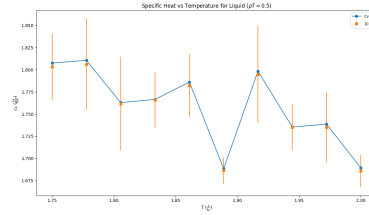


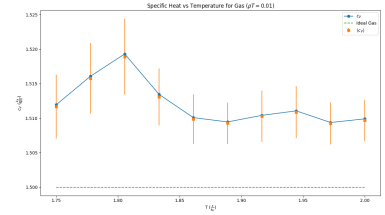
Figure 9: Plot containing specific heat per atom for all phases of matter as well as the expected behaviour for ideal gas and solid. Here, the $\langle c_V \rangle$ is computed as the mean of the block bootstrapped values with the errors computed by block bootstrap



(a) Solid



(b) Liquid



(c) Gas

Figure 10: Contains plots for the individual phases. c_V is the specific heat per atom computed while $\langle c_V \rangle$ is computed as the mean of the block bootstrapped values with the errors computed by the block bootstrap method

are not sure since we do not know how the critical temperature varies with pressure, and also because we do not know the exact pressure in our system.

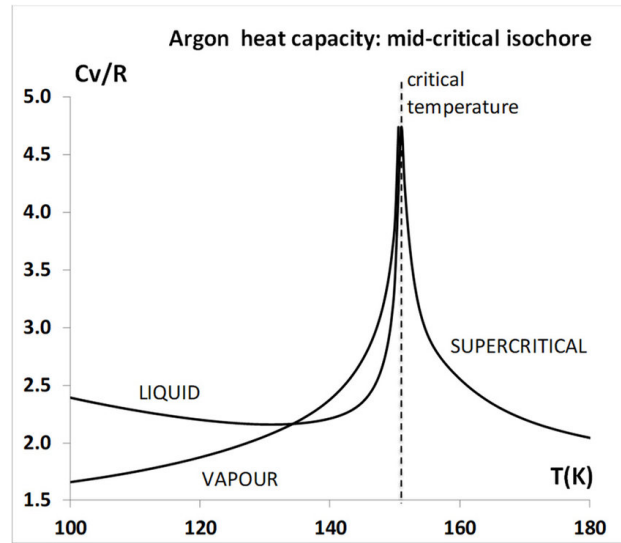


Figure 11: Specific heat of argon out of a paper by L. Woodcock[7]

4 Conclusion

In this project, we successfully implemented a molecular dynamics simulation of Argon, capturing its phase behaviour using the Lennard-Jones potential. The Velocity-Verlet algorithm ensured energy conservation and velocity rescaling ensured the system obtained the desired equilibrium temperature. Observables like Mean Squared Displacement and Specific Heat per atom provided insight into the phase transition process. By systematically varying system parameters and employing automated phase classification, we constructed a phase diagram that roughly aligns with theoretical expectations, although in some regions we observe errors likely due to the limited simulation time.

Performance analysis revealed that the computational bottlenecks lay in force calculations and thermodynamic averaging. While parallelization and caching significantly reduce time requirements while running the code for noting observables, further optimizations—such as GPU acceleration—could be explored in future work. Our code scales with $O(n^2)$ which limits us from running simulations for longer durations. Additionally, refining the classification models for phase detection could slightly enhance accuracy of the estimation.

Overall, our work demonstrates the effectiveness of molecular dynamics in studying phase transitions and highlights the importance of computational efficiency in large-scale simulations.

References

- [1] M. Wimmer and A. Akhmerov, *Computational physics*, Lecture notes for Computational Physics at TUDelft, 2020-2025.
- [2] T. Gowers, J. Barrow-Green, and I. Leader, *The Princeton Companion to Mathematics*. Princeton University Press., 2008, pp. 726–728.
- [3] E. Zohravi, E. Shirani, A. Pishevar, and H. Karimpour, “Dissipative particle dynamics study of velocity autocorrelation function and self-diffusion coefficient in terms of interaction potential strength,” *Molecular Physics*, vol. 116, pp. 1–11, Mar. 2018. DOI: 10.1080/00268976.2018.1441463.
- [4] D. Bolmatov, M. Zhernenkov, D. Zav’yalov, S. Tkachev, A. Cunsolo, and Y. Cai, “The frenkel line: A direct experimental evidence for the new thermodynamic boundary,” *Scientific Reports*, Nov. 2015. DOI: 10.1038/srep15850.

- [5] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "Cupy: A numpy-compatible library for nvidia gpu calculations," in *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. [Online]. Available: http://learningsys.org/nips17/assets/papers/paper_16.pdf.
- [6] J. Thijssen, *Computational Physics*, 2nd ed. Cambridge University Press, 2007.
- [7] L. Woodcock, "Thermodynamics of gas-liquid colloidal equilibrium states: Hetero-phase fluctuations," *Entropy*, vol. 21, p. 1189, Dec. 2019. DOI: 10.3390/e21121189.

5 Appendix

5.1 Configurations

In this section we've included all the configurations necessary to create the various figures in this document.

5.1.1 Euler and Verlet

This is the configuration for the simulator used to generate the energy plots for both Fig. 1 and Fig. 2.

```
{
  "particles": 108,
  "box": {
    "x": 7.78324257,
    "y": 7.78324257,
    "z": 7.78324257
  },
  "time_steps": 1000,
  "step_size": 0.005,
  "equilibrium_steps": 0,
  "temperature": 0.6,
  "temperature_tolerance": 0.1,
  "lattice_const": 2,
  "equilibrium_stable_check": 3,
  "position_method": "fcc",
  "velocity_method": "mbdist",
  "seed": 21921,
  "simulator_type": [
    "verlet",
    "euler"
  ],
  "outputs": [
    "energies"
  ],
  "do_caching": true,
  "export_csv": true
}
```

5.1.2 MSD fixing

This is the configuration for the simulator used to generate the MSD for Fig. 4 and Fig. 5. To create the first of the two figures line 184 of code/simulators.py needs to have % box_dim appended at the end.

```
{
  "particles": 500,
  "box": {
    "x": 10,
    "y": 10,
    "z": 10
  },
  "time_steps": 4000,
}
```

```

"step_size": 0.01,
"equilibrium_steps": 25,
"temperature": 1,
"temperature_tolerance": 0.2,
"equilibrium_stable_check": 2,
"position_method": "fcc",
"velocity_method": "mbdist",
"seed": 2911,
"simulator_type": [
    "verlet"
],
"outputs": [
    "energies",
    "MSD"
],
"do_caching": false,
"export_csv": true
}

```

5.1.3 MSD phase compare

The configurations for the simulator used to generate the subfigures in Fig. 6,. Config for solid

```

{
    "particles": 500,
    "box": {
        "x": 8,
        "y": 8,
        "z": 8
    },
    "time_steps": 2000,
    "step_size": 0.01,
    "equilibrium_steps": 25,
    "temperature": 1,
    "temperature_tolerance": 0.2,
    "equilibrium_stable_check": 2,
    "position_method": "fcc",
    "velocity_method": "mbdist",
    "seed": 29113,
    "simulator_type": [
        "verlet"
    ],
    "outputs": [
        "energies",
        "MSD"
    ],
    "do_caching": true,
    "export_csv": true
}

```

Config for liquid

```

{

```

```

    "particles": 500,
    "box": {
        "x": 10,
        "y": 10,
        "z": 10
    },
    "time_steps": 2000,
    "step_size": 0.01,
    "equilibrium_steps": 25,
    "temperature": 1,
    "temperature_tolerance": 0.2,
    "equilibrium_stable_check": 2,
    "position_method": "fcc",
    "velocity_method": "mbdist",
    "seed": 29113,
    "simulator_type": [
        "verlet"
    ],
    "outputs": [
        "energies",
        "MSD"
    ],
    "do_caching": true,
    "export_csv": true
}

```

Config for gas

```

{
    "particles": 500,
    "box": {
        "x": 50,
        "y": 50,
        "z": 50
    },
    "time_steps": 2000,
    "step_size": 0.01,
    "equilibrium_steps": 25,
    "temperature": 1,
    "temperature_tolerance": 0.2,
    "equilibrium_stable_check": 2,
    "position_method": "fcc",
    "velocity_method": "mbdist",
    "seed": 29113,
    "simulator_type": [
        "verlet"
    ],
    "outputs": [
        "energies",
        "MSD"
    ],
    "do_caching": true,
    "export_csv": true
}

```

}

5.1.4 Results

For both the phase diagram as well as the specific heat per atom, while we do not use the config file, we set the following values directly in the Python file:

- `num_particles = 500`
- `seed = 21921 np.random.seed(seed)`
- `timesteps = 4000`
- `step_size = 0.005`
- `equilibrium_steps = 25`
- `temperature_tolerance = 0.1`
- `equilibrium_stable_check = 3`
- `simulator_type = "verlet"`
- `bin_size = 0.1`

From ρT and T we can compute ρ and thus get volume per particle ($1/\rho$). Using this we can compute the volume ($\text{num_particles} \times \text{volume per particle}$) and get the box dimensions ($\text{volume}^{1/3}$), and lattice constant ($M/\text{box dimension where num_particles} = 4M^3$).